Auth

Auth And Access Low-Level Design

1. Document Control

Item	Value
System	ITIL Group Reports Automation
Component	Authentication, Authorisation, and Admin Users
Owner	CNE Automation / ITIL Reporting
Status	QA implementation active
Last updated	2026-07-14

2. Purpose

Auth and Access protects operational APIs, manages local application users, records login/session activity, and enforces role-based access for report, workspace, division-admin, and system-admin functions.

3. Scope

In scope:

- Login, logout, current-user, and password-change endpoints.
- Super Admin bootstrap.
- User creation, status, role, delete, password reset, sessions, and login history.
- Role-based dependencies used by routers.
- Temporary-password enforcement.
- Error envelope behavior for auth-related failures.

4. Runtime Components

Component	Path	Responsibility
Auth core	`api/auth.py`	Password hashing, JWTs, sessions,

		bootstrap, role dependencies
Auth router	`api/routers/auth.py`	Auth and admin-user endpoints
App startup	`api/app.py`	Runs auth bootstrap when API starts
Error handlers	`api/errors.py`	Standard API error envelope

5. Roles And Access Levels

Role	Purpose
`SUPER_ADMIN`	System owner; user administration, sessions, run logs, and all admin actions
`ADMIN`	Operational admin; division/group maintenance where allowed
`MANAGER`	Elevated operational user
`USER`	Standard authenticated user
`READ_ONLY`	View-only authenticated user

Key access rules:

- Daily report reads are available to authenticated users.
- Division/group mapping changes require `ADMIN` or `SUPER_ADMIN`.
- Run logs, user admin, login history, and session revocation require `SUPER_ADMIN`.
- `SUPER_ADMIN` cannot be assigned through normal create-user or role-update payloads.

6. Bootstrap

On startup, `api/app.py` calls `ensure_super_admin_user()` unless startup DB requirements are relaxed for a non-production environment.

Variable	Purpose

`SUPER_ADMIN_EMAIL`	Configured system-owner account
`SUPER_ADMIN_INITIAL_PASSWORD`	Initial password used only when creating the Super Admin
`AUTH_ALLOWED_EMAIL_DOMAINS`	Allowed email domains for account creation/bootstrap
`JWT_SECRET`	Token signing secret
`REQUIRE_DB_ON_STARTUP`	Controls whether startup fails when DB is unavailable

The configured Super Admin email is normalized by trimming surrounding quotes and lowercasing.

7. Auth Endpoints

```
POST /api/auth/login
POST /api/auth/logout
GET  /api/auth/me
POST /api/auth/change-password

```

Behavior:

- Login validates email, password, account status, and deleted flag.
- Successful login creates a session and access token.
- Logout revokes the current session.
- `/auth/me` restores the current authenticated user from the token.
- Change password requires the current password and a new password of at least 12 characters.
- Users with `must_change_password = true` may access `/auth/me`, `/auth/logout`, and `/auth/change-password`, but role-protected operational APIs are blocked until the password is changed.

8. Admin User Endpoints

All endpoints require `SUPER_ADMIN`.

```
GET    /api/admin/users
POST   /api/admin/users
DELETE /api/admin/users/{user_id}
PATCH  /api/admin/users/{user_id}/status
PATCH  /api/admin/users/{user_id}/role
POST   /api/admin/users/{user_id}/reset-password
GET    /api/admin/login-history
```

```
GET    /api/admin/sessions
DELETE /api/admin/sessions/{session_id}

```

Rules:

- New users require `email`, `full_name`, `password`, and `role`.
- User-created passwords must be at least 12 characters.
- Created and reset users can be forced through `must_change_password`.
- `SUPER_ADMIN` is reserved for the configured system-owner account.
- Deleting users is soft-delete behavior.
- Status values are `ACTIVE`, `DISABLED`, and `LOCKED`.
- Session revocation records audit context.

9. Error Handling

All errors use the standard API envelope from `api/errors.py`.

Scenario	Error behavior
Invalid email or password	`401`, `INVALID_CREDENTIALS`
Current password incorrect	`401`, `CURRENT_PASSWORD_INCORRECT`
Temporary password still active	`PASSWORD_CHANGE_REQUIRED`
Missing or invalid token	Authentication error
Insufficient role	Authorization error
Database unavailable	`503` with support `request_id`

Frontend must show the backend `message` and keep `request_id` visible/copyable for support.

10. Testing Strategy

Smoke tests:

```
POST /api/auth/login
GET /api/auth/me
POST /api/auth/change-password
POST /api/auth/logout
GET /api/admin/users
POST /api/admin/users
PATCH /api/admin/users/{user_id}/status
POST /api/admin/users/{user_id}/reset-password
GET /api/admin/sessions
```

```
DELETE /api/admin/sessions/{session_id}

```

Expected checks:

- Super Admin bootstrap succeeds with valid env and reachable DB.
- Temporary-password users are blocked from role-protected endpoints.
- `ADMIN` can mutate division mappings but cannot manage users.
- `SUPER_ADMIN` can manage users and sessions.
- Disabled or locked users cannot log in.
- API error responses include `message`, `error_code`, and `request_id`.

11. Acceptance Criteria

Auth and Access is accepted when users can sign in, restore sessions, change password, and sign out; Super Admin bootstrap is repeatable; protected routers use correct dependencies; and auth failures are clear enough for frontend support.